STEP 1 OF THE PROCESS · FROM 0 TO HERO

DISNEY AGENT SUPPORT TICKETS

Before writing a single prompt in v0, prepare these 6 folders. This brief is now fully answered across all 11 sections — your first job is reading it closely (it locks in a CLI-first build, not a dashboard), not filling in blanks. This guide gives you a starting structure grounded in the real answers.

14/20 — HIGH · 6 folders · Ticket clustering · Root-cause ranking · Resolution playbooks · Synthetic data only

Now fully briefed — scored 14/20, HIGH. All 11 sections are answered (Pain 5 · Visibility 2 · Prototypability 5 · Fit 4 · Effort −2), ported from the delivery team's own detailed notes. **Key call: this is scoped as a CLI tool with live Jira integration, not a web dashboard** — read brief §5 before assuming otherwise. Still genuinely open: the client-side decision-maker and a client-side SME to route hackathon-week questions to (both TBD in the brief) — reach out to **Daniela Martin** in the meantime. A synthetic support-ticket sample is in `04_Data_Sources/ticket-sample.json`.

1 WHAT WE KNOW SO FAR

The one-liner: a CLI-first agent that ingests a team's support-ticket backlog from Jira, clusters it into root-cause incidents instead of a flat list of "High Priority" noise, and hands back a ready-to-use resolution playbook — built for any support/DevOps team, not just Disney.

The pain: in the last two years, Disney's DevOps team has burned ~80% of its time on support tickets, leaving almost no room for innovation; the team is also training an offshore team to absorb some of the load. The opportunity is to flip that ratio.

The user: exceptionally technical SRE/DevOps engineers who live in the terminal — a CLI is the right format, not a dashboard.

Delivery lead: **Luciano**. Product owner: Ivan Rueda. R&D owner: Mich Galvez.

2 KEY BUILD CALLS LOCKED IN BY THE BRIEF

Straight from §5 and §11 — these decide the shape of the build, so don't re-litigate them without a reason:

- **CLI, not GUI.** Three terminal commands: `learn` (ingest history) → `triage` (categorize + override priority inflation) → `solve` (print the playbook). No web dashboard is in scope.
- **Live Jira, not mocked.** The CLI should connect live to a dev/sandbox Jira board via an API token — Jira is the only source in scope (no easy ServiceNow access).
- The lean-in moment is the CLI turning a chaotic wall of "High Priority" tickets into one clean root-cause incident with a trustworthy playbook — build for that, not for the deflection step.
- Still open: who watches/approves on the client side (decision-maker + SME are both TBD) — flag this rather than guessing a name.

3 THE 6 FOLDERS — A STARTING STRUCTURE

01_Context — Business context

`04_Disney_Agent Support Tickets/01_Context/`

⚠ **AI-generated starter, built from the fully-answered brief.** Confirm the offshoring detail and exact ticket volume with Luciano before treating this as final.

Without it, v0 builds a generic "ticket triage" tool. With real DevOps team context, it feels like Disney's actual 80%-on-tickets problem.

- The 80% support / near-0% innovation ratio the DevOps team lives with today — the team is also training an offshore team to absorb some of the load
- Illustrative volume: ~120-150 tickets/week feeding the queue — confirm the real number with the client-side SRE once assigned
- The users: SRE/DevOps engineers who triage tickets today — exceptionally technical, terminal-native, which is why the build is a CLI, not a dashboard

```
create: disney-context.md
```

Source: brief §1–§4 (fully answered)

02_BRD_PRD_Specs — Requirements

`04_Disney_Agent Support Tickets/02_BRD_PRD_Specs/`

⚠ **AI-generated starter, built from the fully-answered brief.** Confirm the real ticket volume with the client-side SRE before treating the numbers as final.

§5 and §11 already give you the shape — this folder is about writing it down cleanly, not discovering it.

- BRD: freeing up DevOps time for innovation is the business case — flip the 80%-support ratio via MTTR reduction and ticket-volume dedup
- PRD: three CLI commands — `learn` (ingest history), `triage` (categorize + override priority inflation), `solve` (print the playbook) — live Jira integration, no GUI

```
create: BRD.md
```

```
create: PRD.md
```

Source: brief §5, §11 (fully answered)

03_UXUI — Design system

`04_Disney_Agent Support Tickets/03_UXUI/`

⚠ **AI-generated starter, built from the fully-answered brief.** The brief is explicit that this is CLI-first, not a dashboard — don't default to a web UI here.

The brief's users are "exceptionally technical, terminal-native" — a CLI is the deliberate choice, not a shortcut.

- Think terminal, not dashboard: clean, readable CLI output for `learn` / `triage` / `solve` — a structured RCA printout, not a UI mockup
- Disney logo/branding is not required for a CLI tool — keep this folder light

```
create: cli-output-style.md
```

```
Disney_Logo.webp (kept for reference only — not used in a CLI)
```

Source: brief §4, §5 (fully answered)

04_Data_Sources — Synthetic dataset

`04_Disney_Agent Support Tickets/04_Data_Sources/`

⚠ **AI-generated starter sample — already added.** A synthetic 15-ticket sample is in this folder to get you moving; the brief calls for a 50-100 ticket real sample pulled from the assigned Jira board once a client-side SRE is confirmed — swap it in if you can.

§6 and §11 are explicit that this must be synthetic/anonymized for the demo (no live client data) — the sample below already follows that rule.

- A synthetic batch of 15 support tickets across 5 realistic clusters (CI/CD pipeline timeout, API 502s, post-renewal auth failure, email deliverability, low-value feature requests) — already in `ticket-sample.json`
- Scale this up to 50-100 tickets for a more convincing clustering/ranking demo if time allows
- No real client/PII data without clearance — flag and default to synthetic

```
ticket-sample.json (added — 15 synthetic tickets, 5 clusters)
```

Source: brief §11 + this session's synthetic sample

05_APIs — What's mocked vs. live

`04_Disney_Agent Support Tickets/05_APIs/`

⚠ **AI-generated starter, built from the fully-answered brief.** The brief is explicit here — confirm the dev/sandbox Jira token is actually available before build day.

§7 and §11 already answer this: Jira, live.

- Ticket source: **Jira** — the team doesn't have easy access to ServiceNow, so Jira is the only source in scope
- For the demo, connect LIVE to a dev/sandbox Jira board via an API token — don't mock it. The clustering/RCA/playbook logic runs on an LLM backend (e.g. Amazon Bedrock or similar)

```
create: README.md
```

Source: brief §7, §11 (fully answered)

06_Features — Prioritized list

`04_Disney_Agent Support Tickets/06_Features/`

⚠ **AI-generated starter, built from the fully-answered brief.** The three CLI commands below are the brief's actual scope — build them in order.

§5 gives you the exact three commands and the explicit out-of-scope line — no GUI, no auto-execution of fixes.

- P0: `learn` — ingest historical ticket/resolution data
- P0: `triage` — categorize incoming tickets, override priority inflation, calculate real impact
- P1: `solve` — print the root-cause analysis + step-by-step resolution playbook
- Out of scope: any GUI/dashboard, and any auto-execution of fixes into production

```
create: features-disney.md
```

Source: brief §1, §5 (fully answered)

4 BUILD THE MASTER PROMPT, THEN ITERATE IN V0

Once the brief's gaps are filled and the 6 folders reflect real answers, feed them to **Claude, Gemini, or ChatGPT** to draft your Master Prompt — stack, tone, ground rules (synthetic data) — **no code yet**. Build in validated chunks: (0) Masterprompt, (1) Foundation + ticket data, (2) clustering + root-cause ranking (P0), (3) resolution playbook generation (P1), (4) iterate. Keep the number of prompts small.